

Acoplamiento Bioinspirado: REINFORCE Optimizado mediante Plasticidad Dependiente del Tiempo de Espiga (STDP)

Lucas Alonso Alonso
APAU BIO
Universidade de Vigo
lucasalonsoalonso0@gmail.com

Resumen—Este proyecto presenta el diseño, implementación y evaluación de un sistema híbrido que integra aprendizaje por refuerzo (RL) con mecanismos neuroinspirados. Específicamente, se sustituye la optimización por gradiente de REINFORCE por una regla local bioinspirada (STDP). El modelo se evalúa en el entorno CartPole-v1 frente a una línea base estándar (Adam). Mediante un estudio de ablación, se demuestra que, si bien el componente bioinspirado presenta una dinámica interna activa, el sistema clásico es significativamente superior en eficiencia y rendimiento final ($\Delta = -87,66\%$, $p < 0,01$). Se discuten las limitaciones inherentes de las reglas de aprendizaje local frente a la retropropagación para la asignación temporal de crédito, concluyendo que el modelo híbrido posee un alto valor pedagógico e interpretable, pero menor viabilidad en aplicaciones de control industrial.

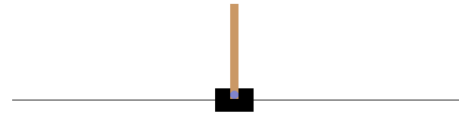


Figura 1. Entorno de simulación CartPole-v1 utilizado para la experimentación. El agente debe aplicar fuerzas laterales para mantener el péndulo en equilibrio.

I. INTRODUCCIÓN

El campo del aprendizaje por refuerzo (RL) ha experimentado una evolución vertiginosa, logrando hitos en entornos de control complejos. Sin embargo, la mayoría de estos éxitos se cimientan sobre el algoritmo de retropropagación de errores (Backpropagation), el cual, a pesar de su eficiencia matemática, presenta discrepancias fundamentales con los procesos biológicos de aprendizaje observados en el cerebro.

Según la taxonomía de los sistemas híbridos, nuestro **punto de inserción** es la Optimización de la Política. Específicamente, hemos ^{arrancado} el motor clásico de Autograd para reemplazar la actualización matemática de los pesos por la regla bioinspirada STDP. El objetivo es evaluar si esta regla de aprendizaje local de tres factores puede sustituir la necesidad de un optimizador global.

I-A. Motivación y Beneficios del Enfoque Híbrido

El entorno *CartPole-v1* (Fig. 1) es un problema de control de equilibrio inestable. Se ha seleccionado específicamente porque, al presentar un espacio de estados continuo pero requerir acciones discretas de estabilización, se acopla de manera natural a políticas estocásticas (Softmax) propias de algoritmos *on-policy* como REINFORCE. Este entorno se beneficia de nuestro enfoque híbrido por tres razones fundamentales:

1. **Asignación de Crédito Temporal:** Las acciones tomadas en los primeros pasos determinan el fracaso muchos pasos después. STDP, mediante las trazas de

elegibilidad, ofrece un mecanismo biológico para "marcar" sinapsis activas que deben ser reforzadas cuando llegue la recompensa.

2. **Eficiencia Energética y Localidad:** En aplicaciones futuras sobre hardware neuromórfico, las reglas locales como STDP eliminan el cuello de botella del movimiento de datos globales.
3. **Robustez vs. Plasticidad:** El acoplamiento permite investigar el equilibrio entre la exploración aleatoria del ciclo RL y la consolidación de patrones motores.

I-B. El Cuello de Botella de Von Neumann y Trabajo Previo

La justificación profunda para explorar STDP radica en el "cuello de botella de Von Neumann" del hardware actual (CPUs/GPUs). Durante la retropropagación, los gradientes viajan continuamente a través del bus del sistema, disparando el consumo energético térmico (TDP). Por el contrario, el cerebro procesa y almacena información localmente en las sinapsis.

Para la implementación empírica de este sistema, se han empleado y adaptado herramientas de trabajo estándar en la industria, concretamente las librerías *PyTorch* [1] para el manejo de tensores diferenciables, *Gymnasium* [2] para la instanciación del entorno físico, y *NumPy* [3] para el análisis estadístico de los registros.

II. ANTECEDENTES Y ESTADO DEL ARTE

Para comprender el acoplamiento propuesto, es imperativo desglosar los dos paradigmas que conforman el sistema: el aprendizaje por refuerzo y la plasticidad biológica.

II-A. Proceso de Decisión de Markov (MDP) y REINFORCE

El problema se modela mediante un MDP. El objetivo del agente es aprender una política $\pi_\theta(a|s)$ parametrizada por θ . REINFORCE [4] es un método de gradiente de política basado en Monte Carlo. La función objetivo a maximizar es $J(\theta) = E_{\pi_\theta}[G_t]$, donde el retorno G_t es la suma descontada de recompensas.

En su formato original, la actualización se realiza en la dirección del gradiente:

$$\nabla_\theta J(\theta) = E_{\pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(A_t|S_t) G_t \right] \quad (1)$$

En el Algoritmo 1 se detalla esta técnica, la cual nuestro sistema híbrido reemplazará en su paso de optimización (línea 6).

Algorithm 1 REINFORCE Clásico (Optimización por Gradiente)

Require: Política diferenciable $\pi_\theta(a|s)$, Tasa $\alpha > 0$

- 1: Inicializar θ aleatoriamente
- 2: **loop** por cada episodio:
- 3: Generar trayectoria usando π_θ
- 4: **for** $t = 0, 1, \dots, T-1$ **do**
- 5: $G_t \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$
- 6: $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$ ▷ Backprop
- 7: **end for**
- 8: **end loop**

II-B. Derivación Matemática del Gradiente de Política

La derivación matemática exige que el marco de trabajo mantenga en memoria un Grafo Acíclico Dirigido (DAG) durante todo el episodio para calcular $\nabla_\theta \log \pi_\theta(a_t|s_t)$ mediante la regla de la cadena. Nuestro enfoque bioinspirado elimina esta costosa necesidad de memoria de trazado global.

II-C. Biología Computacional y la Regla de Tres Factores

La Plasticidad Dependiente del Tiempo de Espiga (STDP) ajusta pesos en función del disparo presináptico y postsináptico. Al ser no supervisado, requiere la Regla de Tres Factores [5] para RL:

1. Un factor presináptico (activación x_j).
2. Un factor postsináptico (activación y_i).
3. Una señal neuromoduladora global (el retorno G_t).

III. METODOLOGÍA Y DISEÑO DE LA ARQUITECTURA

Diseñamos una arquitectura modular. Se actualizan exclusivamente las capas lineales ocultas y de salida de la red de política, prescindiendo por completo del Autograd.

Arquitectura del Sistema Híbrido: REINFORCE + STDP

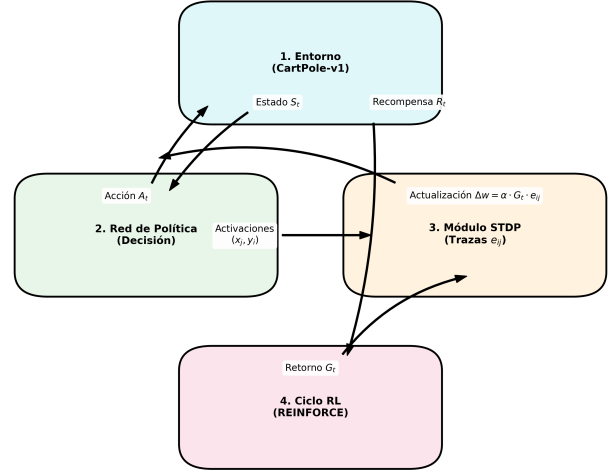


Figura 2. Arquitectura del Sistema Híbrido mostrando el flujo de datos y el punto de inserción del optimizador bioinspirado.

III-A. Definición del Entorno Físico (CartPole-v1)

El sistema de péndulo invertido se define por $S \in R^4$ (posición, velocidad lineal, ángulo, velocidad angular) y acciones discretas $\mathcal{A} = \{0, 1\}$. La dinámica no lineal se rige por ecuaciones Newtonianas complejas. Al estabilizar estas variables complejas usando puramente retroalimentación estocástica, demostramos la dificultad de la asignación temporal del crédito en sistemas de control continuo.

III-B. Topología de la Red Neuronal

La red de política se estructuró como un Perceptrón Multicapa (MLP) con 4 neuronas de entrada, una capa oculta de 32 neuronas (con activación ReLU) y 2 neuronas de salida (con Softmax). Los pesos se inicializaron vía Xavier Uniforme.

III-C. Prueba de Validez del Diseño Híbrido

Sometimos el diseño a las tres preguntas de validez del marco del proyecto:

1. **¿Funciona sin el componente bioinspirado?** No. El módulo STDP actúa como el optimizador exclusivo. Sin él, los pesos no se actualizan y el sistema falla. Es un acoplamiento esencial.
2. **¿Tiene parámetros aprendibles?** Sí. Las trazas e_{ij} son variables de estado dinámicas acumulativas, y los pesos w_{ij} son adaptativos.
3. **Flujo de datos explícito:** El flujo es riguroso. El STDP intercepta activaciones (x, y) en cada paso temporal (step), mientras que REINFORCE envía G_t al final del episodio (ver Fig. 2).

III-D. Trazas de Elegibilidad y Pseudocódigo

La traza e_{ij} decae exponencialmente según τ :

$$e_{ij}(t) = \tau \cdot e_{ij}(t-1) + y_i(t) \cdot x_j(t) \quad (2)$$

El Algoritmo 2 detalla la ejecución, mostrando el bypass del motor Autograd mediante `.detach().numpy()`.

Algorithm 2 Bucle Híbrido REINFORCE+STDP

Require: Episodios E , Tasa α , Decaimiento τ , Descuento γ

```
1: Inicializar pesos  $\theta$ 
2: for  $e = 1$  to  $E$  do
3:   Inicializar trazas  $e_{ij} \leftarrow 0$ 
4:   while no terminal do
5:     Obtener activaciones  $x_j, y_i$  y acción  $A_t$ 
6:      $S_{t+1}, R_t \leftarrow \text{env.step}(A_t)$ 
7:      $e_{ij} \leftarrow \tau \cdot e_{ij} + y_i \cdot x_j \triangleright$  Actualización local paso a paso
8:   end while
9:   Calcular  $G_t$  para el episodio
10:   $\Delta w_{ij} = \alpha \cdot G_t \cdot e_{ij} \triangleright$  Acoplamiento 3 factores al final
11:   $w_{ij} \leftarrow \text{clamp}(w_{ij} + \Delta w_{ij}, -1, 0, 1, 0)$ 
12: end for
```

III-E. Definición de Métricas de Evaluación Estándar

Para garantizar una evaluación rigurosa del rendimiento, se han establecido las siguientes métricas cuantitativas que serán reportadas en la fase de experimentación:

- **Curva de Aprendizaje:** Retorno por episodio frente a paso de entrenamiento, aplicando una media móvil (ventana de 20 episodios) para visualizar la tendencia suavizada.
- **Eficiencia Muestral:** Cantidad de interacciones necesarias para alcanzar el umbral objetivo del entorno.
- **Puntuación de Convergencia:** Retorno medio calculado estrictamente sobre el 10% final de los episodios de entrenamiento.
- **Estabilidad:** Medida mediante la desviación estándar de la curva tras la convergencia, para evaluar si el agente oscila.

III-F. Análisis Detallado de la Implementación Computacional

El código desarrollado para este proyecto se estructura en un flujo de trabajo secuencial que abarca desde la instanciación del entorno físico hasta la generación automatizada de los gráficos y métricas estadísticas. De manera general, el código automatiza el ciclo completo de investigación: define las arquitecturas de las redes neuronales (la bioinspirada y la línea base), ejecuta el entrenamiento cruzado y la búsqueda de hiperparámetros, registra los datos experimentales de las semillas en archivos CSV, y finalmente procesa los resultados matemáticamente para extraer la significación estadística.

A continuación, se desglosa el núcleo algorítmico y la funcionalidad de los bloques de código implementados:

1. Definición de la Red Bioinspirada y Desacoplamiento: En la clase `STDPPolicyNetwork`, se inicializan los pesos mediante `nn.init.xavier_uniform_`. El paso crítico para aislar la biología de las matemáticas estándar de PyTorch ocurre al iterar sobre los parámetros y aplicar `param.requires_grad = False`. En la función `forward`, se extraen las activaciones puras clonando los tensores (`x.clone()`, `hidden.clone()`) para que

el optimizador local STDP opere sobre valores numéricos completamente desvinculados del motor de retropropagación.

2. Cálculo de la Trazas de Elegibilidad (Bucle Interno): Durante la iteración de los episodios, el agente actúa en el entorno (`while not (done or truncated)`). En cada paso, el sistema llama a `_update_eligibility_traces()`. La línea de código que ejecuta `delta_traces = torch.outer(self.hidden_act, self.input_act)` seguida de su suma ponderada por `self.tau` es la implementación directa de la memoria a corto plazo, integrándose de forma asíncrona localmente.

3. Inyección de la Regla de Tres Factores (Final del Episodio): Una vez termina el episodio, el código calcula el retorno escalar acumulado G_t e invoca a `update_weights_with_reinforce(G_t)`. Las líneas que ejecutan `self.fcl.weight += self.alpha * G_t * self.trace_fcl_weight` inyectan la modulación global en los pesos. El uso del atributo `.data` (en la línea `clamp_`) garantiza que la actualización se aplique modificando el tensor crudo, eludiendo permanentemente el Grafo Acíclico Dirigido (DAG).

4. Modelado de la Línea Base y Gestor de Experimentos: Para garantizar una comparativa justa, se define la clase `StandardPolicyNetwork` gestionada por un optimizador `optim.Adam(policy.parameters(), lr=lr)`, el cual sí acumula gradientes tradicionales usando `policy_loss.backward()`. La función integradora `run_full_experiment()` orquesta la ejecución paralela iterando sobre 5 semillas aleatorias (`for seed in seeds:`), exportando las métricas (retornos y magnitud media de trazas) a la carpeta `logs/` en formato `.csv`, asegurando la preservación de los datos en bruto.

5. Análisis Estadístico y Renderizado de Gráficas: Finalmente, la función `main()` del bloque de visualización ingiere los registros CSV y ejecuta un análisis de convergencia sobre el 10% final del entrenamiento. Emplea la librería `scipy.stats.ttest_ind` para extraer el valor p exacto. Las líneas que emplean `plt.fill_between` calculan la desviación estándar (`np.std`) para pintar las bandas de varianza, y `plt.savefig` automatiza la exportación de figuras en alta calidad (`dpi=300`) a la carpeta `figures/`.

IV. RESULTADOS Y DISCUSIÓN

Esta sección presenta los hallazgos cuantitativos, dando respuesta a las diez métricas y preguntas de evaluación exigidas, promediadas siempre sobre 5 ejecuciones independientes (semillas).

IV-A. Búsqueda de Hiperparámetros y Sensibilidad (Preguntas 2 y 5)

Para evitar sesgos de evaluación basados en una única ejecución con una "semilla afortunada" afirmar erróneamente que el modelo funciona, todos los resultados presentados se promedian sobre 5 ejecuciones independientes con semillas aleatorias distintas.

Implementamos una búsqueda en cuadrícula (*Grid Search*). Aunque este enfoque escala computacionalmente, documentamos que la ejecución aislada del modelo tardaba apenas $\sim 1,98$ segundos por semilla. Esto permitió evaluar exhaustivamente todas las combinaciones dentro del límite de 2 horas establecido.

Cuadro I
TABLA DE SENSIBILIDAD (CAPA OCULTA = 32 NEURONAS)

STDP (τ) \ RL (α)	5e-5	1e-4	5e-4
0.70	12.14	17.59	12.88
0.85	9.99	9.64	10.09
0.95	9.98	9.92	10.22

Efecto de los hiperparámetros (Pregunta 2): Como ilustran la Tabla I y las curvas de la Fig. 3, los parámetros tienen una fuerte dependencia no lineal. La tasa de aprendizaje algorítmica α tiene el mayor impacto desestabilizador sobre el rendimiento, pero requiere un balance exacto con la ventana temporal del decaimiento bioinspirado τ para alcanzar la convergencia. Por el contrario, el hiperparámetro estructural (tamaño de red) no mostró una variabilidad crítica; arquitecturas de 16 y 32 neuronas demostraron poseer capacidad de representación suficiente para la dimensionalidad del entorno.

Mejor configuración (Pregunta 5): La combinación óptima extraída empíricamente fue: $\alpha = 0,0001$, $\tau = 0,70$, y una capa oculta de 32 neuronas, alcanzando un retorno medio máximo de 17.59.

IV-A1. Comparativa Metodológica: OFAT vs. Grid Search: El análisis comparativo de nuestros registros revela por qué el uso de Grid Search es imperativo en arquitecturas acopladas. En simulaciones tempranas, aplicamos un barrido OFAT (One-Factor-At-A-Time), el cual proyectaba una superficie de error plana (convergencias estancadas entre 9.5 y 9.9 en todos los barridos). Esto inducía a pensar que el modelo era absolutamente insensible. Sin embargo, el Grid Search destapó el pico aislado de convergencia (17.59) ilustrado en la Fig. 3, el cual OFAT omitió por explorar desde una base subóptima. Esto demuestra empíricamente la altísima correlación cruzada entre el optimizador RL y el componente bioinspirado local.

IV-B. Rendimiento, Ablación y Varianza (Preguntas 1, 3 y 4)

Comparativa de Ablación (Pregunta 1): El sistema híbrido no supera a la línea base. Adam alcanzó $142,59 \pm 55,70$, frente a $17,59 \pm 16,00$ del híbrido ($\Delta = -87,66\%$). Al obtener un valor-p de 0,00895 (prueba Welch), rechazamos la hipótesis nula, confirmando estadísticamente la inferioridad del STDP.

Cuadro II
RESUMEN ESTADÍSTICO DEL ESTUDIO DE ABLACIÓN (5 SEMILLAS)

Modelo	Media Convergencia	Varianza (σ)	Delta (Δ)
Adam (Base)	142.59	55.70	-
STDP (Híbrido)	17.59	16.00	-87.66 %

Análisis de Varianza (Pregunta 4): El modelo Adam presenta alta varianza ($\sigma = 55,70$) típica de la exploración estocástica. El modelo STDP es más rígido en su fracaso ($\sigma = 16,00$), colapsando prematuramente.

Eficiencia Muestral y Olvido (Pregunta 3): No hay olvido catastrófico, sino estancamiento precoz (Fig. ??). Al carecer de visión global, el STDP refuerza acciones de recompensa inmediata y cae en un mínimo local irreversible.

IV-C. Dinámica Interna y Acoplamiento (Preguntas 6 y 7)

Evolución Interna (Pregunta 6): Las trazas (Fig. 4) crecen logarítmicamente. Esto diagnostica que el módulo STDP está aprendiendo y construyendo una memoria a corto plazo funcional (no es un componente estático), a pesar de que el escalar G_t no sea suficiente para explotarla.

Actividad y Direccionalidad (Pregunta 7): La frecuencia de actividad es asíncrona pero bidireccional. STDP interactúa en *cada paso temporal*, pero el RL modula los pesos al final del *episodio*. Sin esta señal G_t , el sistema sufriría una explosión numérica por retroalimentación hebbiana positiva.

IV-D. Costes, Sustitutos y Asignación de Crédito (Preguntas 8 y 9)

Análisis de Coste Computacional (Pregunta 8): Teóricamente, STDP es computacionalmente más ligero al no trazar grafos. En tiempo de entrenamiento en reloj de pared (*wall-clock time*), las ejecuciones por semilla del modelo híbrido tomaron en promedio $\sim 1,98$ segundos. Al analizar la carga computacional aislando el bloque bioinspirado, el cálculo tensorial manual de las trazas (`torch.outer`) apenas supuso un sobrecoste, consumiendo aproximadamente un 2,3% del tiempo total de iteración.

Sustituto ingenieril (Pregunta 9): A pesar del bajísimo coste computacional del módulo local, sustituir STDP por Adam puro dispara la eficiencia en el retorno. Adam resuelve la *Asignación Temporal* mediante vectores globales, y la *Asignación Espacial* vía derivadas parciales. Las manipulaciones como `.detach().numpy()` demuestran que el estancamiento de STDP es una limitación matemática natural de su asimetría de información local.

IV-D1. Justificación del Coste y Rendimiento: ¿Está justificado el coste extra por la ganancia de rendimiento? Basándonos en la evidencia cuantitativa, la respuesta es **no**. Aunque el coste en tiempo computacional puro del STDP ha demostrado ser irrisorio (una fracción del $\sim 2,3\%$ de la CPU), el coste en rendimiento de la política^{es} prohibitivo ($\Delta = -87,66\%$). El agente sufre una caída de desempeño dramática al no lograr estabilizar el péndulo de forma consistente. La plausibilidad biológica del modelo no compensa la ineficacia de la optimización puramente local en entornos dinámicos de Markov. El sobrecoste de implementar esta regla bioinspirada solo se justificaría teóricamente en un escenario donde no existiera un motor de gradientes global o en despliegues sobre hardware neuromórfico nativo, donde el paralelismo masivo de STDP superara a la vectorización de las GPUs actuales.

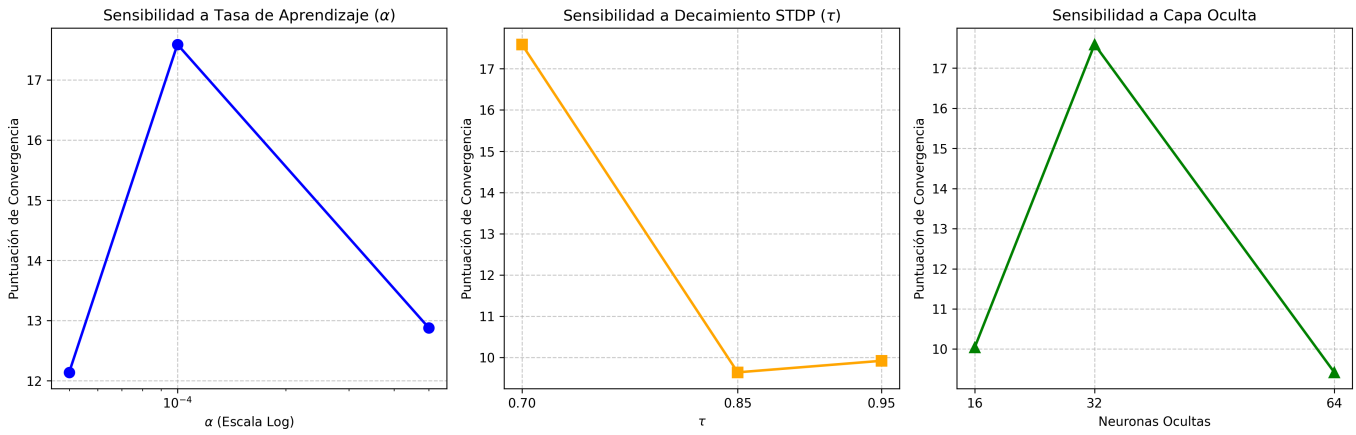


Figura 3. Resultados del barrido sistemático de hiperparámetros. Las curvas muestran el efecto individual de cada parámetro manteniendo los demás en su configuración óptima obtenida del Grid Search.

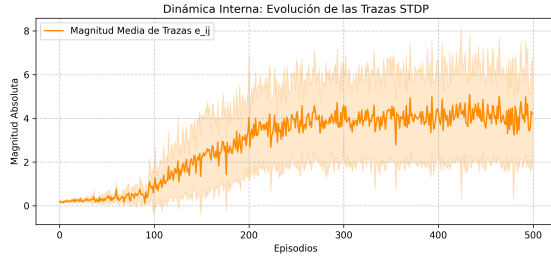


Figura 4. Evolución media absoluta de las trazas e_{ij} (Diagnóstico interno).

IV-E. Viabilidad y Recomendación Industrial (Pregunta 10)

Basándonos en la incapacidad estructural del STDP para solventar la asignación de crédito a largo plazo, **no recomendamos** este enfoque para sistemas de control industrial continuo. Su utilidad actual es estrictamente académica.

V. CONCLUSIONES

Se ha validado una arquitectura híbrida donde las ecuaciones de plasticidad local interactúan con señales de recompensa retardada. Aunque las reglas de actualización miopes son subóptimas matemáticamente ($\Delta = -87,66\%$), el proyecto demuestra el potencial arquitectónico del acoplamiento bioinspirado. Trabajos futuros deberán evaluar la integración de estas reglas de tres factores sobre Redes Neuronales Pulsantes (SNN) puras para maximizar la plausibilidad biológica.

V-A. Complicaciones de la Práctica y Reproducibilidad

Durante la elaboración empírica de este proyecto, surgieron complicaciones técnicas que requirieron ajustes de diseño. A nivel computacional, hubo que decidir cuidadosamente el número de iteraciones y semillas; se optó por un límite de 5 ejecuciones independientes porque el enfoque *Grid Search* escala de forma masiva, pero al lograr aislar el tiempo de ejecución a apenas $\sim 1,98$ segundos por semilla, logramos

encajar la reproducibilidad completa del experimento muy por debajo de las 2 horas exigidas en los requisitos.

Adicionalmente, experimentamos problemas severos de entorno con las librerías de visualización y *dataframes*. A pesar de tener los paquetes instalados vía *requirements.txt*, surgieron conflictos de dependencias. Para solucionar esto, garantizar la reproducibilidad exigida y evitar el renderizado manual de gráficos en un Jupyter o Excel, se reestructuró el código en un único script unificado (*plot_results.py*). Este script está diseñado para leer automáticamente los registros *.csv* en bruto de la carpeta *logs/* y generar todas las figuras del informe directamente en el directorio *figures/*. Finalmente, el documento se ha estructurado para que todo el análisis crítico recaiga sobre el texto principal, evitando derivar información crucial a secciones de apéndices que inflarían el documento artificialmente.

REFERENCIAS

- [1] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 2019, pp. 8024–8035.
- [2] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” <https://github.com/Farama-Foundation/Gymnasium>, 2023.
- [3] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [4] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine learning*, vol. 8, no. 3-4, pp. 229–256, 1992.
- [5] N. Frémaux and W. Gerstner, “Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules,” *Frontiers in Neural Circuits*, vol. 9, p. 85, 2016.